K-DBSCAN

Philip Loewen, Arusan Surendiran

DBSCAN Algorithm

This section defines the manual implementation of the DBSCAN algorithm. It utilizes a queue in data structure and a pairwise distance matrix to handle cluster expansion efficiently in R.

https://www.geeksforgeeks.org/dsa/basic-operations-for-queue-in-data-structure/

```
manual_dbscan <- function(data, eps, minPts) {
  n <- nrow(data)
  cluster_assignment <- rep(0, n)  # 0 = unvisited, -1 = noise
  cluster_id <- 0

  # Compute pairwise Euclidean distances to define epsilon-neighborhoods
  dist_mat <- as.matrix(dist(data, method = "euclidean"))

  for (i in 1:n) {
    # Skip point if it has already been visited
    if (cluster_assignment[i] != 0) next

    # Retrieve the epsilon-neighborhood for point i
    neighbors <- which(dist_mat[i, ] <= eps)

    # Check core point condition
    if (length(neighbors) < minPts) {
      cluster_assignment[i] <- -1 # Temporarily classify as a noise point
    } else {
      # Core point verified: create a new density-based cluster
      cluster_id <- cluster_id + 1
      cluster_assignment[i] <- cluster_id

      # Identify directly density-reachable points to start the chain
      seeds <- setdiff(neighbors, i)
```

```r
    # Initialize a queue to establish density-connected points
    queue <- integer(n)
    queue_len <- length(seeds)

    if (queue_len > 0) {
      queue[1:queue_len] <- seeds
    }

    head <- 1
    tail <- queue_len

    # Expand the cluster to satisfy the Maximality requirement
    while (head <= tail) {
      current_p <- queue[head]
      head <- head + 1

      # If previously classified as noise, it is a border point
      if (cluster_assignment[current_p] == -1) {
        cluster_assignment[current_p] <- cluster_id
      }

      # If unvisited, process for density-reachability
      if (cluster_assignment[current_p] == 0) {
        cluster_assignment[current_p] <- cluster_id

        # Retrieve the epsilon-neighborhood of the current point
        new_neighbors <- which(dist_mat[current_p, ] <= eps)

        # If this point is also a core point, expand the reachable chain
        if (length(new_neighbors) >= minPts) {
          # Extract only unvisited or noise points
          valid_new <- new_neighbors[cluster_assignment[new_neighbors] <= 0]
          num_new <- length(valid_new)

          if (num_new > 0) {
            queue[(tail + 1):(tail + num_new)] <- valid_new
            tail <- tail + num_new
          }
        }
      }
    }
  }
```

```r
  }
  return(cluster_assignment)
}
```

Datasets

This section handles the retrieval and preprocessing of the real-world datasets from the UCI Machine Learning Repository.

```r
library(remotes)
#remotes::install_github("coatless-rpkg/ucimlrepo")
library(ucimlrepo)
library(here)
```

```
here() starts at /Users/arusansurendiran/Home/My GitHub/K-DBSCAN
```

```r
# Abalone Dataset
abalone_uci <- fetch_ucirepo(name = "abalone")
abalone <- abalone_uci$data$original
colnames(abalone) <- c("Sex", "Length", "Diameter", "Height", "Whole_w",
                       "Shucked_w", "Viscera_w", "Shell_w", "Rings")

# Prepare and scale Abalone for clustering
abalone_numeric <- abalone[, 2:8]
abalone_scaled <- scale(abalone_numeric)

# MAGIC Gamma Telescope Dataset
MAGIC_uci <- fetch_ucirepo(id=159)
magic <- MAGIC_uci$data$original


# Shuttle Data

shuttle_train <- read.table("data/shuttle.trn", header = FALSE, sep = " ")
shuttle_test <- read.table("data/shuttle.tst", header = FALSE, sep = " ")
statlog_shuttle <- rbind(shuttle_train, shuttle_test)

# Separate features from labels, the last column (V10) is the class label
statlog_features <- statlog_shuttle[, 1:9]
```

Simulation

The simulation evaluates the algorithm's performance on non-linear geometries (sine curves, rings, and arcs) and its ability to distinguish clusters from background noise. We demonstrate the advantages of DBSCAN clustering in comparison to k-means clustering.

```
library(dbscan)
```

```
Attaching package: 'dbscan'


The following object is masked from 'package:stats':

    as.dendrogram
```

```
library(ggplot2)
library(MASS)
set.seed(2201)

# Define the cluster centers and sizes
centers <- matrix(c(25, 25, -25, -25, 25, -20, -20, 20), ncol = 2, byrow = TRUE)
sizes <- c(1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 250, 100)

# Generate synthetic clusters with different characteristics
cluster1 <- mvrnorm(n = sizes[1], mu = centers[1,], Sigma = matrix(c(9, 2, 2, 9), ncol = 2))
cluster2 <- mvrnorm(n = sizes[2], mu = centers[2,], Sigma = matrix(c(12, 0.5, 0.5, 4), ncol =
cluster3 <- mvrnorm(n = sizes[3], mu = centers[3,], Sigma = matrix(c(6, 0.3, 0.3, 18), ncol =
cluster4 <- mvrnorm(n = sizes[4], mu = centers[4,], Sigma = matrix(c(15, 0.1, 0.1, 15), ncol

# Cluster 5: Sine curve distribution
x_values <- seq(-40, -20, length.out = 1000)
y_values <- sin(x_values) * 2 + rnorm(1000, mean = 0, sd = 0.2) # Adding some noise to the si
cluster5 <- cbind(x_values, y_values)

# Cluster 6 & 7: Intersecting clusters
cluster6 <- mvrnorm(n = 1000, mu = c(-5, -5), Sigma = matrix(c(7, 1.3, 1.3, 8), ncol = 2))
cluster7 <- mvrnorm(n = 1000, mu = c(-3, -7), Sigma = matrix(c(7, 1.3, 1.3, 5), ncol = 2))

# Cluster 8: Points along an arc
theta <- runif(1000, min = pi, max = 2 * pi)
r <- 5
```

```r
cluster8 <- cbind(r * cos(theta) + 5, r * sin(theta) + 15)

# Cluster 9: Points in a "moon" shape
theta <- runif(1000, min = 0, max = pi)
r <- 10 + runif(1000, min = -1, max = 1)
cluster9 <- cbind(r * cos(theta) + 30, r * sin(theta))

# Cluster 10: Points in a "ring" shape around all other clusters
theta <- runif(1000, min = 0, max = 2 * pi)
r_inner <- 48
r_outer <- 52
r <- sqrt(runif(1000, min = r_inner^2, max = r_outer^2))
cluster10 <- cbind(r * cos(theta), r * sin(theta))

# Noise and Outliers
noise <- cbind(runif(250, -60, 60), runif(250, -60, 60))
outliers <- cbind(rnorm(100, 0, 20), rnorm(100, 0, 20))

data_full <- rbind(cluster1, cluster2, cluster3, cluster4, cluster5,
                   cluster6, cluster7, cluster8, cluster9, cluster10, noise, outliers)
df_full <- as.data.frame(data_full)
colnames(df_full) <- c("X", "Y")
```

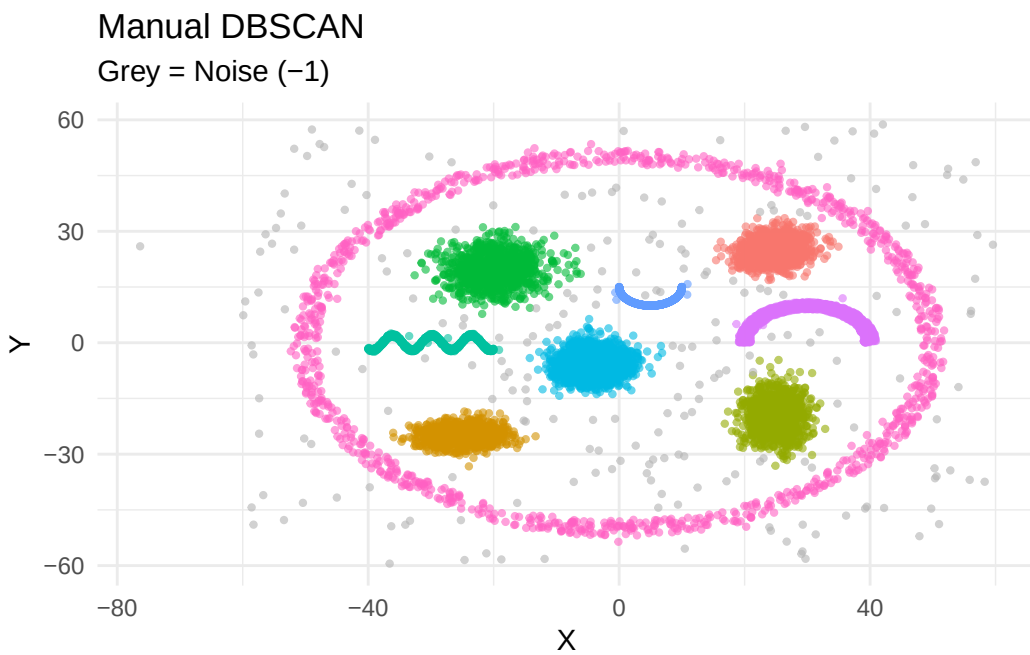
Execution and Comparison

```r
set.seed(2201)
# Run Manual DBSCAN
results_dbscan <- manual_dbscan(df_full, eps = 2.5, minPts = 4)
df_full$Cluster <- as.factor(results_dbscan)

# Run K-means for Comparison
km_result <- kmeans(df_full, centers = 10, nstart = 25)
df_full$Kmeans_Cluster <- as.factor(km_result$cluster)
```

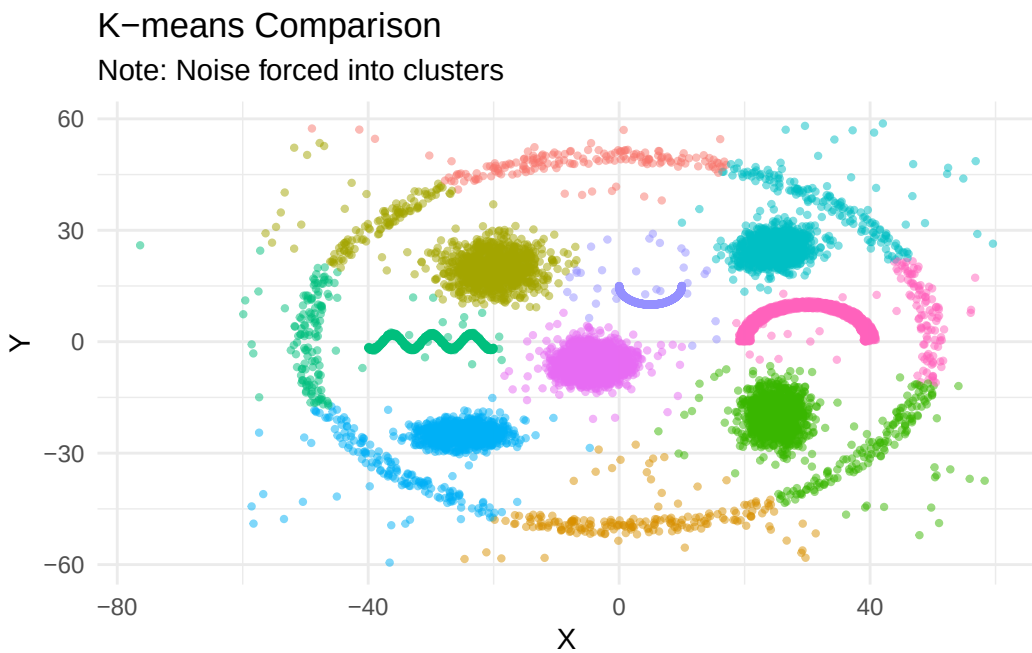
Visualization

```r
# Extract unique clusters excluding the noise (-1)
db_clusters <- unique(results_dbscan[results_dbscan != -1])
```

```r
db_colors <- setNames(scales::hue_pal()(length(db_clusters)), db_clusters)
db_palette <- c("-1" = "grey70", db_colors)


# DBSCAN Results
ggplot(df_full, aes(x = X, y = Y, color = Cluster)) +
  geom_point(alpha = 0.6, size = 0.8) +
  scale_color_manual(values = db_palette) +
  theme_minimal() +
  labs(title = "Manual DBSCAN", subtitle = "Grey = Noise (-1)") +
  theme(legend.position = "none")
```




```r
# K-means Results
ggplot(df_full, aes(x = X, y = Y, color = Kmeans_Cluster)) +
  geom_point(alpha = 0.5, size = 0.8) +
  theme_minimal() +
  labs(title = "K-means Comparison", subtitle = "Note: Noise forced into clusters") +
  theme(legend.position = "none")
```



Application of Algorithm

K-means++ Algorithm

K-means++ improves upon the standard k-means algorithm by providing a specific method for choosing initial cluster centers. Standard k-means often relies on random initialization, which can lead to poor local optima. The k-means++ approach spreads out the initial centers before starting the iterative assignment and update steps.

Implementation

The following implementation includes the probabilistic seeding process and the iterative refinement loop.

```
# Function to compute Euclidean distance between a point and a set of centers
get_min_dist_sq <- function(data, centers) {
  # Calculate squared distances from each point to the nearest existing center
  apply(data, 1, function(point) {
    min(colSums((t(centers) - point)^2))
  })
}
```

```r
manual_kmeans_pp <- function(data, k, max_iter = 100) {
  data_mat <- as.matrix(data)
  n <- nrow(data_mat)

  # --- Step 1: K-means++ Seeding ---

  # Choose the first center uniformly at random
  centers <- matrix(NA, nrow = k, ncol = ncol(data_mat))
  centers[1, ] <- data_mat[sample(1:n, 1), ]

  if (k > 1) {
    for (i in 2:k) {
      # Compute squared distances to the nearest center already chosen
      dist_sq <- get_min_dist_sq(data_mat, centers[1:(i-1), , drop = FALSE])

      # Select next center with probability proportional to D(x)^2
      # This ensures centers are spread out across the feature space
      probs <- dist_sq / sum(dist_sq)
      next_idx <- sample(1:n, 1, prob = probs)
      centers[i, ] <- data_mat[next_idx, ]
    }
  }

  # --- Step 2: Standard K-means Iteration ---

  assignments <- rep(0, n)
  prev_assignments <- rep(-1, n)
  iter <- 0

  while (!all(assignments == prev_assignments) && iter < max_iter) {
    prev_assignments <- assignments
    iter <- iter + 1

    # Assignment Step: Assign each point to the nearest center
    dist_to_centers <- as.matrix(dist(rbind(centers, data_mat)))
    # Extract distances between points and centers only
    dist_matrix <- dist_to_centers[(k + 1):(n + k), 1:k]
    assignments <- apply(dist_matrix, 1, which.min)

    # Update Step: Recompute centers as the mean of assigned points
    for (j in 1:k) {
      if (sum(assignments == j) > 0) {
```

```
        centers[j, ] <- colMeans(data_mat[assignments == j, , drop = FALSE])
      }
    }
  }

  return(list(cluster = assignments, centers = centers, iterations = iter))
}
```

Simulation and Evaluation

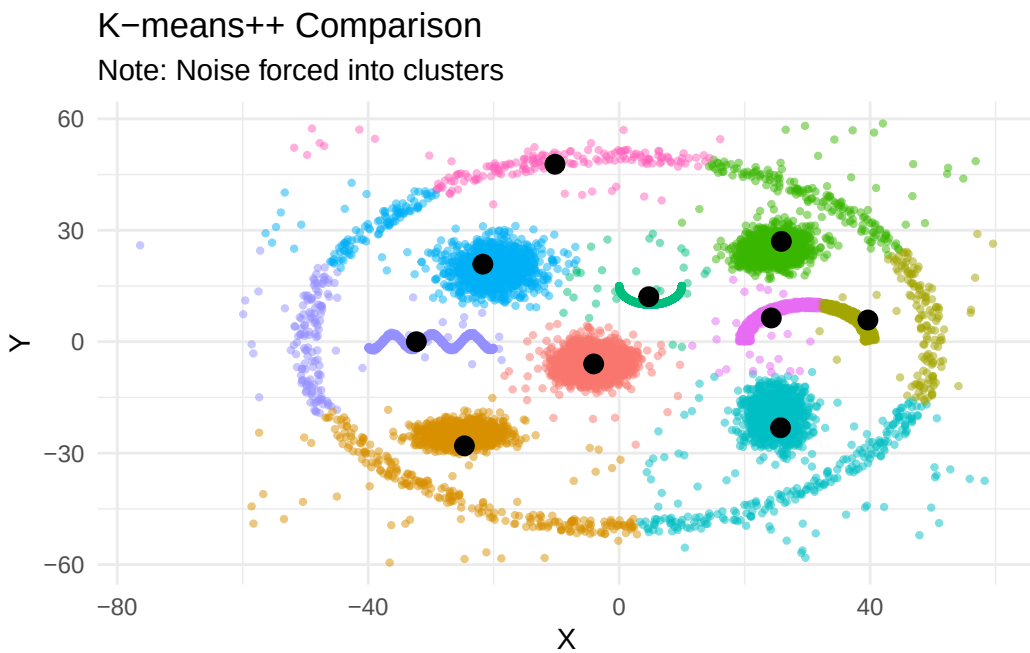
We reuse the previous synthetic dataset to evaluate how effectively k-means++ identifies the underlying structure.

```
set.seed(2201)
# Run Manual K-means++
# We choose k=10 to match the number of intended clusters in the generation
k_clusters <- 10
results_kmpp <- manual_kmeans_pp(df_full[, c("X", "Y")], k = k_clusters)
df_full$Kmpp_Cluster <- as.factor(results_kmpp$cluster)

centers_df <- as.data.frame(results_kmpp$centers)
colnames(centers_df) <- c("X", "Y")
```

Visualization

The plot below displays the final cluster assignments. The larger black points represent the final centroids calculated by the algorithm.

```
# Kmpp-means Results
ggplot(df_full, aes(x = X, y = Y, color = Kmpp_Cluster)) +
  geom_point(alpha = 0.5, size = 0.8) +
  geom_point(data = centers_df, aes(x = X, y = Y),
             color = "black", size = 3) +
  theme_minimal() +
  labs(title = "K-means++ Comparison", subtitle = "Note: Noise forced into clusters") +
  theme(legend.position = "none")
```



Application to Abalone dataset

This section applies the k-means++ implementation to the numeric features of the Abalone dataset.

```
# Assuming the 'abalone_scaled' object exists from previous preprocessing
# If not, ensure the data is loaded and scaled as shown in the DBSCAN section.
if (exists("abalone_scaled")) {
  abalone_results <- manual_kmeans_pp(abalone_scaled, k = 3)
  table(abalone_results$cluster)
} else {
  print("Scaled Abalone data not found. Please run the preprocessing steps.")
}
```

```
   1    2    3
1165 1253 1759
```

K-DBSCAN Algorithm

The primary goal of the K-DBSCAN hybrid approach is to process data in localized segments and then unify density-connected clusters that cross the boundaries of those segments. By

partitioning the space with K-means, the algorithm only performs exhaustive distance checks between points within the same group or between adjacent groups.

Implementation

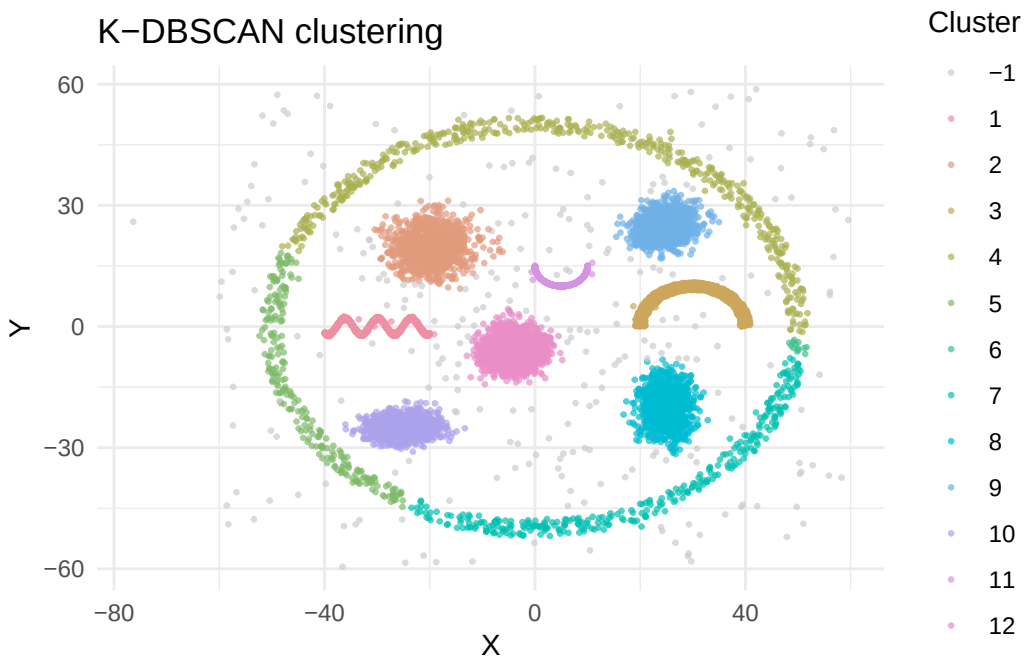
The following implementation integrates the `manual_dbscan` and `manual_kmeans_pp` methods.

```
single_link_dist <- function(clu_1, clu_2) {
  clu_1 <- as.matrix(clu_1)
  clu_2 <- as.matrix(clu_2)

  if (nrow(clu_1) == 0 || nrow(clu_2) == 0) {
    return(Inf)
  }

  dist_mat <- as.matrix(dist(rbind(clu_1, clu_2)))
  n1 <- nrow(clu_1)
  n2 <- nrow(clu_2)
  return(min(dist_mat[1:n1, (n1 + 1):(n1 + n2), drop = FALSE]))
}

cluster_dist <- function(clu_1, clu_2) {
  clu_1 <- as.matrix(clu_1)
  clu_2 <- as.matrix(clu_2)

  if (nrow(clu_1) == 0 || nrow(clu_2) == 0) {
    return(Inf)
  }

  center_1 <- colMeans(clu_1)
  center_2 <- colMeans(clu_2)

  dis_KCij <- sqrt(sum((center_1 - center_2)^2))
  dis_ri <- max(sqrt(rowSums((clu_1 - matrix(center_1, nrow = nrow(clu_1), ncol = ncol(clu_1
  dis_rj <- max(sqrt(rowSums((clu_2 - matrix(center_2, nrow = nrow(clu_2), ncol = ncol(clu_2

  return(dis_KCij - dis_ri - dis_rj)
}


find_uf <- function(parent, x) {
```

```r
  while (parent[x] != x) {
    x <- parent[x]
  }
  return(x)
}

union_uf <- function(parent, x, y) {
  rx <- find_uf(parent, x)
  ry <- find_uf(parent, y)
  if (rx != ry) parent[rx] <- ry
  return(parent)
}

k_dbscan <- function(D, k, max_iter, eps, minPts) {
  # Step 1: Normalization of D
  D_mat <- as.matrix(scale(D))
  n <- nrow(D_mat)

  if (n == 0) {
    return(integer(0))
  }

  # Step 2: Data grouping with K-means Algorithm
  km_results <- manual_kmeans_pp(D_mat, k = k, max_iter = max_iter)
  group_id <- km_results$cluster

  # Step 3: Run DBSCAN inside each K-means group
  group_points <- vector("list", k)
  group_clusters <- vector("list", k)
  numclu <- integer(k)
  group_centers <- matrix(NA_real_, nrow = k, ncol = ncol(D_mat))

  for (i in seq_len(k)) {
    group_idx <- which(group_id == i)
    if (length(group_idx) == 0) {
      group_points[[i]] <- list()
      group_clusters[[i]] <- integer(0)
      next
    }

    group_centers[i, ] <- colMeans(D_mat[group_idx, , drop = FALSE])
    group_data <- D_mat[group_idx, , drop = FALSE]
```

```r
  local_labels <- manual_dbscan(group_data, eps, minPts)
  local_cluster_ids <- sort(unique(local_labels[local_labels > 0]))

  numclu[i] <- length(local_cluster_ids)
  group_points[[i]] <- lapply(local_cluster_ids, function(local_cluster) {
    group_idx[local_labels == local_cluster]
  })
  group_clusters[[i]] <- rep(0L, numclu[i])
}

# Step 4: Merge border clusters
# total number of sub-clusters across all groups
total_subclu <- sum(numclu)

# build a flat index: which group and local index does sub-cluster s belong to?
# sub_id[g][c] gives the global sub-cluster index for group g, local cluster c
sub_id <- vector("list", k)
counter <- 0L
for (g in seq_len(k)) {
  sub_id[[g]] <- integer(numclu[g])
  for (c in seq_len(numclu[g])) {
    counter <- counter + 1L
    sub_id[[g]][c] <- counter
  }
}

parent <- seq_len(total_subclu)

# merge border clusters
if (k > 1) {
  for (a in seq_len(k - 1L)) {
    for (b in (a + 1L):k) {
      if (numclu[a] == 0L || numclu[b] == 0L) next

      dis_k <- cluster_dist(D_mat[group_id == a, , drop = FALSE],
                            D_mat[group_id == b, , drop = FALSE])
      if (dis_k >= eps) next

      for (c in seq_len(numclu[a])) {
        for (d in seq_len(numclu[b])) {
          dis_db <- single_link_dist(
            D_mat[group_points[[a]][[c]], , drop = FALSE],
```

```
            D_mat[group_points[[b]][[d]], , drop = FALSE]
          )
          if (dis_db <= eps) {
            parent <- union_uf(parent, sub_id[[a]][c], sub_id[[b]][d])
          }
        }
      }
    }
  }
}

# Step 5: Run DBSCAN again inside each flagged component
# find unique components by their root
all_roots <- unique(vapply(seq_len(total_subclu), find_uf, integer(1), parent = parent))

final_labels <- rep(-1L, n)
final_cluster_id <- 0L

for (root in all_roots) {
  # collect all original points belonging to this component
  component_points <- integer(0)

  for (g in seq_len(k)) {
    for (c in seq_len(numclu[g])) {
      if (find_uf(parent, sub_id[[g]][c]) == root) {
        component_points <- c(component_points, group_points[[g]][[c]])
      }
    }
  }

  component_points <- sort(unique(component_points))
  if (length(component_points) == 0) next

  flagged_data <- D_mat[component_points, , drop = FALSE]
  flagged_result <- manual_dbscan(flagged_data, eps, minPts)

  for (local_cluster in sort(unique(flagged_result[flagged_result > 0]))) {
    final_cluster_id <- final_cluster_id + 1L
    final_labels[component_points[flagged_result == local_cluster]] <- final_cluster_id
  }
}
```

```
  return(final_labels)
}
```

```
results_kdbscan <- k_dbscan(df_full[, c("X", "Y")], 10, 4, 0.1, 4)
df_full$kdbs_cluster <- factor(results_kdbscan)

ggplot(df_full, aes(x = X, y = Y, color = kdbs_cluster)) +
  geom_point(size = 0.5, alpha = 0.7) +
  scale_color_manual(
    values = c("-1" = "grey80", setNames(hcl.colors(length(unique(results_kdbscan[results_kdb
                                                as.character(sort(unique(results_kdbscan[results_kdb
  ) +
  labs(title = "K-DBSCAN clustering", color = "Cluster") +
  theme_minimal()
```